

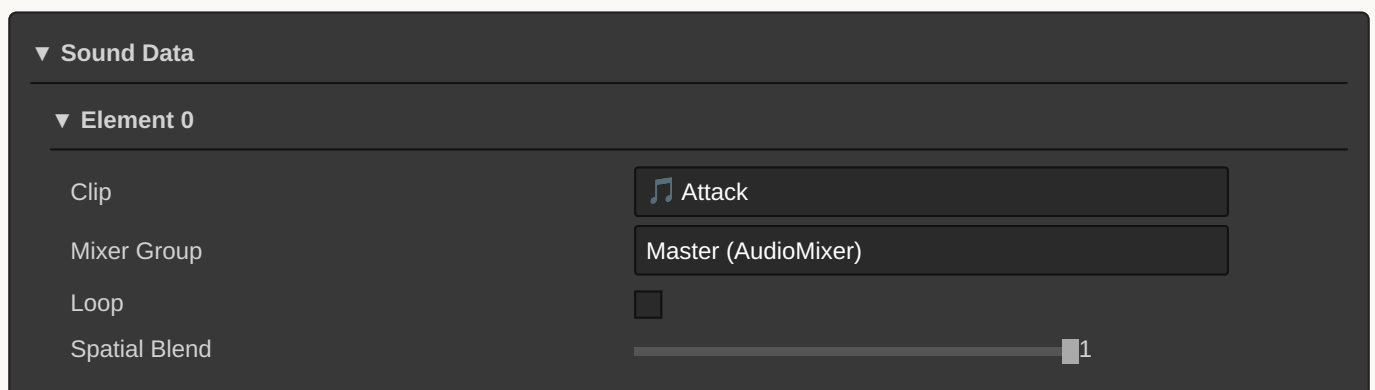
GUÍA DE AUDIO PARA GAME DESIGNERS

Objetivo: Este documento está diseñado para que cualquier miembro del equipo pueda agregar, configurar y reproducir sonidos en el juego fácilmente, sin necesidad de saber programar sistemas complejos.

Hemos implementado un sistema modular (SoundManager) que nos ayuda a mejorar el rendimiento del juego y mantener el código ordenado. En lugar de poner componentes *AudioSource* por todos lados, usamos un sistema de "encargo" (Builder) y una estructura de datos (SoundData).

1. Configuración en el Inspector (SoundData)

Para crear un sonido nuevo, lo primero que necesitas es exponer una variable de tipo SoundData en tu script. Esto generará un menú en el Inspector de Unity donde podrás ajustar cómo suena sin tocar el código.



2. Cómo reproducirlo por Código (Una sola vez)

Hacer sonar algo de forma rápida es tan simple como esto (ejemplo sacado del ataque cuerpo a cuerpo):

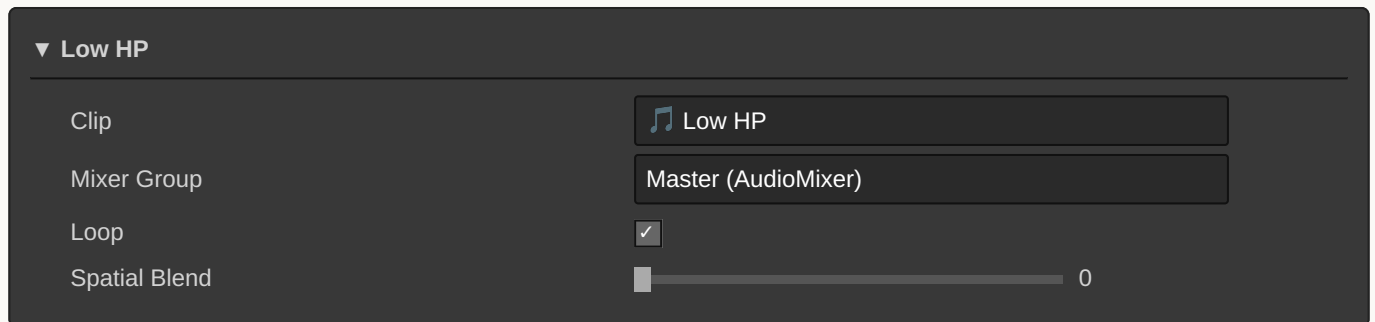
```
if (swingSound != null)
{
    SoundManager.Instance.CreateSound()
        .WithSoundData(swingSound)           // 1. Le pasamos la data del Inspector
        .WithPosition(transform.position)     // 2. Le decimos dónde suena (3D)
        .WithRandomPitch(true)               // 3. Variamos un poco el tono
    (opcional)
        .play();                             // 4. ¡Lo reproducimos!
}
```

En este caso, el sonido se reproduce, termina, y **se limpia solo** (vuelve al pool automáticamente).

3. Sonidos en Bucle (Loop) y Control Manual

Hay situaciones donde queremos que un sonido se repita infinitamente (como una sirena, un aura de veneno, o el **latido del corazón cuando tienes poca vida**). En estos casos, **NO** queremos que el sonido se limpie solo, sino que necesitamos detenerlo nosotros manualmente cuando la acción termine.

Primero, en el Inspector, marcamos la casilla **Loop**:



Luego, en el código, necesitamos guardar una "referencia" a ese sonido mientras está sonando usando la palabra clave `out`, para después poder apagarlo:

```
private SoundEmitter miSonidoEnLoop; // Variable global en tu script para guardar la referencia

void ActivarAlarma()
{
    if (miSonidoEnLoop == null)
    {
        SoundManager.Instance.CreateSound()
            .WithSoundData(_LowHP)
            .play(out miSonidoEnLoop); // ¡USAMOS 'out' PARA GUARDARLO!
    }
}

void ApagarAlarma()
{
    if (miSonidoEnLoop != null)
    {
        // Lo devolvemos manualmente al Pool para que deje de sonar
        SoundManager.Instance.ReturnToPool(miSonidoEnLoop);
        miSonidoEnLoop = null;
    }
}
```

4. Guía Rápida de Uso e Implementación

1. **Abre el script** donde quieras poner el sonido y crea una variable así:

```
[SerializeField] private SoundData miSonidoNuevo;
```

2. **Ve al Inspector de Unity** en el objeto que tiene tu script. Configura tu clip, ajusta el Spatial Blend y decide si lleva Loop.
3. **En el código**, llama al `SoundManager.Instance.CreateSound().play()`. Si es un sonido normal, termina con `.play()`. Si es un sonido en bucle (Loop), usa `.play(out miVariableEmitter)` para guardarlo y deténlo luego con `ReturnToPool(miVariableEmitter)`.

5. Buenas Prácticas para Diseñadores

Por favor, lee esto para no romper los oídos de los jugadores ni el rendimiento del juego.

- **Usa `WithRandomPitch(true)` en sonidos repetitivos:** Si el jugador va a escuchar un espadazo 100 veces por minuto, el pitch aleatorio hace que cada golpe suene ligeramente distinto y se sienta más orgánico.
- **Marca la casilla `Is Frequent` si hay Caos:** En combates rápidos, evita que 10 enemigos reproduzcan el mismo sonido de muerte a la vez saturando el audio.
- **Ajusta siempre el Spatial Blend:** No dejes armas o explosiones en 2D (Spatial Blend en 0), ya que sonarán al máximo volumen en todo el mapa. Resérvalo solo para música, UI o alarmas internas (como el Low HP).
- **Cuidado con los Loops huérfanos:** Si activas un sonido en Loop, asegúrate SIEMPRE de programar en qué momento de la lógica del juego vas a llamar a `ReturnToPool()`. Si el enemigo o el jugador muere y el script se desactiva sin devolver el sonido al pool, el sonido quedará sonando en el vacío para siempre.